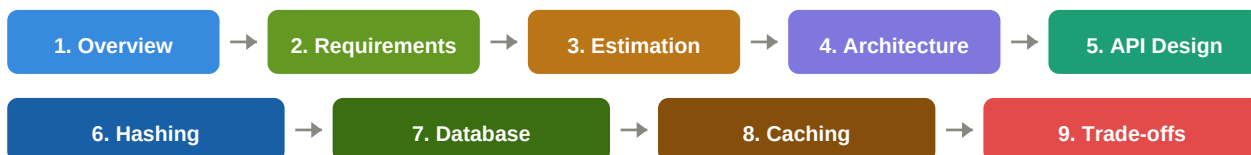


SYSTEM DESIGN SESSION

URL Shortener

Design a system like Bit.ly from scratch



This document is your complete teaching guide. Each section maps to one slide in the presentation. Use the talking points, diagrams, and tables to walk your audience through every concept.

Slide 1 of 9

Slide 1 — Overview: What Are We Building?

A URL shortener takes a long URL and returns a short alias. Visiting the short URL redirects the user to the original. Simple idea — massive engineering challenge.

Input (Long URL)

https://www.example.com/blog/2024/how-to-design-scalable-systems-for-beginners

Output (Short URL)

short.ly/aB3xK9

Why this is a great interview problem

Dimension	Why it matters
Scale	Billions of URLs, millions of redirects per day
Speed	Redirect must complete in <100 ms — latency is UX
Storage	Read-heavy, write-light — perfect for caching
Reliability	A broken short link = broken experience for everyone who shared it

Talking point: Start every system design by restating the problem in one sentence. This shows you understand scope before diving into solutions.

Slide 2 of 9

Slide 2 — Requirements

Always split requirements into Functional (what it does) and Non-Functional (how well it does it). This is the first thing you do in any system design interview.

Functional Requirements

- Shorten a URL — given a long URL, return a unique 6–8 char alias
- Redirect — visiting short URL sends user to original
- Custom alias (optional) — e.g. short.ly/mylink
- Expiry (optional) — links expire after a set period

Non-Functional Requirements

- Availability: 99.99% uptime
- Low latency: redirect <100ms
- Scalability: 100M+ URLs, 10B+ reads/month
- Durability: a short URL must never disappear

Out of scope (for now)

User accounts and dashboards, URL editing/deletion, geographic routing, A/B testing links. Mention these to show you can think broadly, then park them.

Talking point: Clarifying requirements first saves you from designing the wrong system. In interviews, spend 3–5 minutes here before drawing any box.

Slide 3 of 9

Slide 3 — Capacity Estimation

Assume 100M new URLs per month and a 100:1 read-to-write ratio. Back-of-envelope math drives architecture decisions.

~40 / sec
WRITE QPS (URLS CREATED)

~4,000 / sec
READ QPS (REDIRECTS)

~60 GB
STORAGE PER MONTH

~360 GB
BANDWIDTH PER MONTH

How to calculate

Metric	Calculation	Result
Write QPS	100M / (30 days × 86400 sec)	~40 writes/sec
Read QPS	40 × 100 (read:write ratio)	~4,000 reads/sec
Storage/month	100M URLs × 500 bytes avg	~50 GB/month
5-year storage	50 GB × 12 × 5	~3 TB total

Key insight from the numbers

The system is **100x more read-heavy than write-heavy**. This single insight drives the entire architecture — we must optimise for reads. Caching becomes the most important design decision.

URL key space: Base62 with 6 chars = 62^6 = 56 billion unique codes. Enough for any realistic scale.

Talking point: You do not need exact numbers in an interview. Round aggressively and show the method, not the arithmetic.

Slide 4 of 9

Slide 4 — High-Level Architecture

The system handles two flows: Write (shorten) and Read (redirect). Both share the same API layer but differ in what happens next.



Component	Role	Why needed
Load Balancer	Distributes incoming requests across API servers	Single entry point; eliminates hot servers
API Servers	Stateless application logic — shorten & redirect	Stateless = easy horizontal scaling
Redis Cache	In-memory store for hot short → long mappings	Serves 90%+ reads without hitting DB
Primary DB	Persistent store for all URL records	Source of truth; handles writes
Read Replicas	DB copies that serve read queries	Scales reads independently of writes
Analytics Worker	Async processing of click events	Decouples analytics from critical path

Talking point: API servers are **stateless** — they hold no data between requests. This is what makes horizontal scaling easy. Add more servers, put them behind the load balancer, done.

Slide 5 — API Design

Two core REST endpoints cover all functionality. Simple, versioned, JSON-based.

POST /api/v1/shorten

Request body:

```
{ "long_url": "https://example.com/...", "custom_alias": "mylink", "expires_at": "2025-12-31" }
```

Response — 201 Created:

```
{ "short_url": "https://short.ly/aB3xK9", "short_code": "aB3xK9", "expires_at": "2025-12-31" }
```

GET /:short_code

Response — 301 or 302 Redirect:

Location: https://example.com/original/long/path

Status Code	Meaning	When to use
301 Permanent	Browser caches the redirect permanently	When you want to save server load; analytics not needed
302 Temporary	Browser asks server every time	When you need click analytics (most real services use this)

Talking point: The 301 vs 302 choice is a classic trade-off. 301 is more efficient (fewer server hits) but you lose analytics visibility. 302 gives you full click data at the cost of every redirect hitting your servers.

Slide 6 of 9

Slide 6 — URL Shortening Algorithm

The core question: how do we generate a unique 6-character code for every URL? Two approaches.

Approach 1: Hash-based

Hash the long URL with MD5, take the first 6 chars, encode in Base62.

Problem: Two different URLs can produce the same first 6 chars — a collision. Requires a collision-resolution loop.

Approach 2: Counter + Base62 (Recommended)

Maintain a global auto-incrementing counter. Convert each new ID to Base62.

Why better: Guaranteed unique, no collisions, simpler logic. The only concern is the counter being a single point of failure.

Base62 encoding explained

Base62 uses 62 characters: **a–z A–Z 0–9**. Any integer can be converted to this alphabet — the same way decimal (base-10) converts to binary (base-2).

Counter ID	Base62 code	Short URL
1000	qi	short.ly/qi
11157	2TX	short.ly/2TX
56,800,235,583	aaaaaa (6 chars)	short.ly/aaaaaa — 56 billion possibilities

Counter single-point-of-failure fix: Use a distributed ID generator (e.g. Twitter Snowflake) or pre-allocate ranges of IDs to each API server (server 1 gets 1–1M, server 2 gets 1M–2M...).

Talking point: Base62 is the key insight here. Explain it like base-10 → base-2 conversion. Everyone has seen binary — this is the same idea with 62 symbols.

Slide 7 of 9

Slide 7 — Database Design

Simple schema, but important decisions: SQL vs NoSQL, indexing strategy, and how to scale reads.

Schema

urls table / collection:

Column	Type	Notes
short_code	VARCHAR(8)	Primary key — the lookup key for every redirect
long_url	TEXT	The original URL; can be very long
user_id	BIGINT	Owner of the link (nullable for anonymous)
created_at	TIMESTAMP	When the link was created
expires_at	TIMESTAMP	NULL = never expires; TTL job deletes old rows
click_count	BIGINT	Updated by analytics worker asynchronously

SQL vs NoSQL

Factor	SQL (PostgreSQL)	NoSQL (Cassandra/DynamoDB)
Lookup pattern	Works fine for PK lookups	Optimised for exactly this pattern
Horizontal scaling	Complex sharding	Built-in — scales linearly
Transactions	Full ACID	Limited (not needed here)
Recommendation	OK for <10M URLs	Preferred at scale

Talking point: Our access pattern is almost entirely "look up long URL by short_code" — a pure key-value lookup. NoSQL (Cassandra or DynamoDB) is built for exactly this and scales horizontally without complex sharding logic.

Slide 8 of 9

Slide 8 — Caching Strategy

Caching is the single biggest performance win. 80% of traffic hits 20% of URLs — the Pareto principle in action.

Cache-aside (lazy loading) flow

Step	Action	Latency
1	User visits short.ly/aB3xK9 → hits API server	~1 ms
2a Cache HIT	short_code found in Redis → return long URL immediately	~2 ms total
2b Cache MISS	short_code not in Redis → query database	~12 ms
3	Write result back to Redis for future requests	~1 ms
4	Next request for same URL → cache hit	~2 ms

Cache sizing and configuration

Setting	Value	Reason
Cache size	~170 MB	20% of daily URLs × 500 bytes — cheap in RAM
Eviction policy	LRU (Least Recently Used)	Drop the URLs nobody has visited recently
TTL	24 hours (hot links)	Prevents stale cache for updated/deleted URLs
Target hit rate	90%+	Only 10% of reads should reach the database

Talking point: A 90% cache hit rate means your database only handles 400 of the 4,000 reads/sec. Redis can serve a million operations per second. The database never sees the full load.

Slide 9 of 9

Slide 9 — Trade-offs & Design Decisions

Every design decision has a cost. Acknowledging trade-offs is the mark of a senior engineer — not just in interviews.

Decision	Choice Made	Why	Trade-off
ID generation	Counter + Base62	No collisions, simple	Counter = single point of failure
Database type	NoSQL (Cassandra)	Scales horizontally easily	No ACID, limited joins
Redirect code	302 Temporary	Enables click analytics	More server load than 301
Caching	Redis + LRU	Cuts DB load by 90%	Stale data if URL changes
Consistency	Eventual consistency	Acceptable — 1 sec lag is fine	New URLs may not appear instantly worldwide
Availability	Multi-region deploy	High uptime globally	Complex data sync between regions

CAP theorem positioning

URL shorteners favour **Availability over Consistency**. If a new short URL takes 1 second to propagate to all regions, that is perfectly acceptable. But the service must always be up — a down redirect service breaks every page, email, and social post that contains that link.

Extension ideas to mention in your session

- **Rate limiting** — prevent abuse (too many URLs from one IP)
- **Analytics dashboard** — clicks over time, geography, referrers
- **CDN for redirects** — CloudFront edge nodes serve redirects globally in <10ms
- **Bloom filter** — $O(1)$ check if a custom alias is already taken

Talking point: End your session by asking "what would break first as we scale?" The answer is the database — which is why we added caching, replicas, and NoSQL. Telling the story of how the system evolves is more impressive than reciting a perfect architecture from the start.